

Checkpoint 01 Intermediate Report

Camera Calibration Digital Twin

Repository-internal self-contained documentation bundle

final IMU logging fix checkpoint
Analysis run `run_20260406_083611`

Abstract

This report captures the current verified state of the `camera_calibration_digital_twin` repository at Checkpoint 01. It documents the runnable browser-based simulation environment, the fiducial-tag layout, the Pixel 9a-derived camera and IMU model, the offline pose-estimation pipeline, and the numerical results attached to the preserved checkpoint analysis run `run_20260406_083611`. The document is meant to be sendable on its own: it includes the scene description, the relevant model parameters, representative figures copied into the report bundle, and a bibliography positioning the work relative to common camera-calibration and fiducial-tracking references such as Zhang calibration, AprilTag, IPPE-style planar pose estimation, and camera-IMU calibration pipelines [4, 5, 7, 8, 10, 11].

1 Isaac-Oriented Simulation Environment

1.1 Architectural Target and Current Checkpoint Realization

The long-term architectural target of this repository is an Isaac Sim calibration bench: a robot-mounted phone-like sensor rig moving through furnished indoor scenes, publishing camera and IMU outputs, and exposing enough ground truth to validate calibration software end to end. The broader repo structure, API conventions, and module boundaries were designed around that target.

Checkpoint 01 does *not* yet claim that the validated result set comes from a production Isaac runtime. Instead, it preserves the first repository state where the lighter browser-based simulator already reproduces the same core experiment structure cleanly enough to be trusted as a calibration-development environment. That distinction matters, so Table 1 makes it explicit.

Table 1: How the intended Isaac environment maps to the current checkpoint implementation.

Subsystem	Isaac-oriented target	Checkpoint 01 realization
World runtime	Isaac Sim scene, robot, sensors, ROS 2 bridge	browser-friendly synthetic runtime under <code>src/calib_sim/interactive/</code>
Robot layer	articulated manipulator, swappable from config, Isaac articulation control	configurable kinematic arm presets with servo limits and smooth motion presets
Phone rig	RGB camera + IMU rigidly mounted to tool frame	Pixel 9a-style camera model + IMU profile, mounted to the synthetic end-effector
Scene cameras	extra fixed cameras for inspection or logging	observer-camera panels in the browser app
Data capture	synchronized sensor outputs and explicit ground truth	raw MP4, IMU CSV, camera truth CSV, samples JSONL, metadata JSON
Offline analysis	detector + calibration pipeline over recorded data	AprilTag re-detection plus JAX-based modified-Newton fitting

So when this report says “Isaac-oriented environment,” it means: the environment is described the way the final Isaac digital twin needs to be described, but the checkpoint metrics themselves come from the current lightweight realization that already exercises those same abstractions.

1.2 System Role of the Environment

The environment is not just a visual demo. It acts as a deterministic experiment harness for calibration and pose estimation:

- the robot carries a phone-like camera and IMU rig,
- the scene contains long-range wall tags and close-range tabletop tags,
- motion is repeatable and smooth enough to compare solver behavior across runs,
- camera ground truth, tag ground truth, and recorded outputs can be compared directly with fitted results.

That is the same role the future Isaac deployment must serve, even though the graphics and physics backend at Checkpoint 01 is lighter.

1.3 Rendered Environment and Cameras

Figure 1 shows the active checkpoint scene from the main phone view and the three observer views exposed in the web app. This is the easiest visual summary of the environment because it immediately shows the two scale regimes that matter for the estimator: wall-mounted tags in the background and cube-top tags on the table in the foreground.

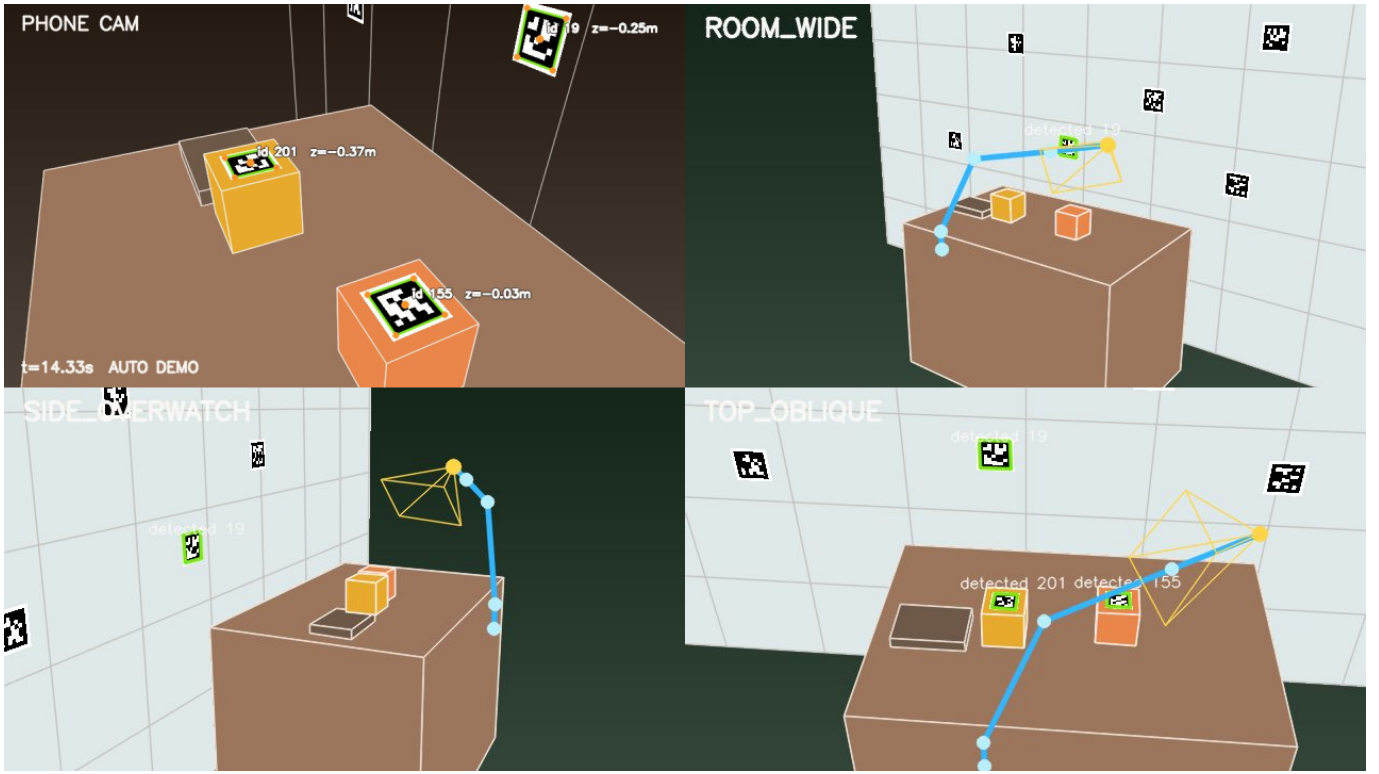


Figure 1: Current checkpoint environment rendered from the simulator. Top-left: primary robot-mounted phone view. Top-right: room-wide observer. Bottom-left: side observer. Bottom-right: top-oblique observer.

Figure 2 complements the montage with a geometry-first top-down schematic. It shows the backdrop wall, table, tabletop props, robot-base location, observer cameras, and the scene look-at reference used by the camera motion logic.

Checkpoint 01 tabletop scene top view

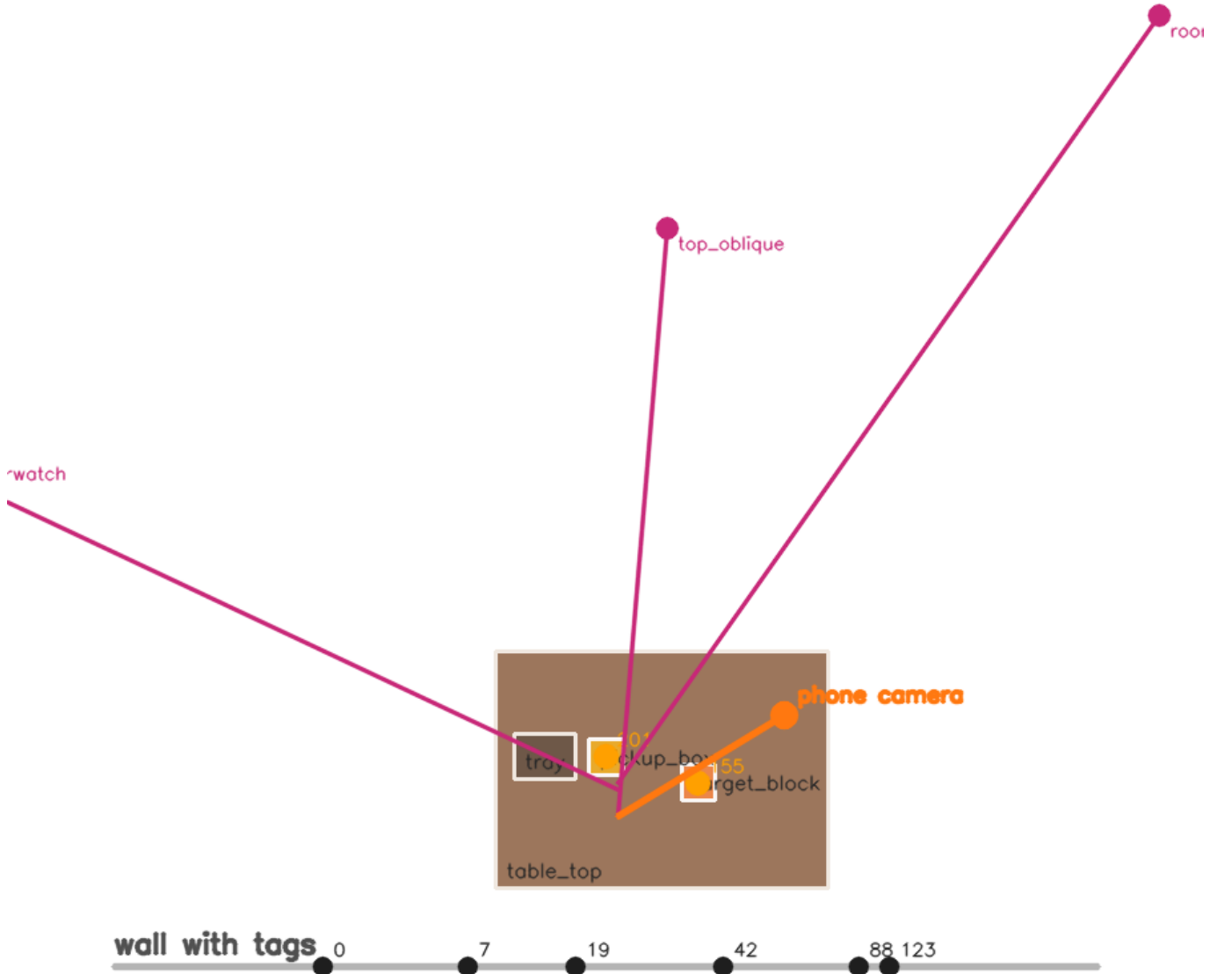


Figure 2: Top-down schematic of the checkpoint scene. The wall with large tags is at $y = 0.000,0$ m. The robot arm base is aligned with the table top height, allowing the phone to move around the tabletop and still look down onto the cube-top targets.

1.4 Scene Geometry

The active environment preset is `config/interactive/tabletop_grab_challenge.yaml`. It is deliberately harder than the earlier bench demo:

- a large wall gives longer-range tag views and wider pose baselines,
- a table and props create near-field clutter,
- two cube-top tags introduce small targets observed at close range and from high angles,
- the combined layout mimics a robot trying to view or approach tabletop objects while still retaining global reference structure.

Table 2: Scene geometry used for Checkpoint 01.

Component	Pose / size	Notes
Backdrop wall	plane at $y = 0.000,0$ m, width 3.200,0 m, height 2.100,0 m	carries the larger wall-mounted tags and sets the far-reference geometry
Table block	center (0.18, 0.60, 0.36) m, size (1.08, 0.72, 0.72) m	top surface at $z = 0.720,0$ m
Target block	center (0.30, 0.56, 0.75) m, size (0.11, 0.11, 0.11) m	carries top-mounted tag 155
Pickup box	center (0.00, 0.64, 0.79) m, size (0.11, 0.11, 0.11) m	carries top-mounted tag 201
Tray	center (-0.20, 0.64, 0.74) m, size (0.20, 0.14, 0.03) m	introduces asymmetry and a manipulation-like destination prop
Primary scene look-at target	(0.04, 0.46, 0.78) m	stabilizes the synthetic phone viewpoint across the demo motion

1.5 Robot Arm and Motion Program

The active arm preset is `config/interactive/robot_arms/long_reach_tabletop.yaml`. Its base position is (0.00, 1.08, 0.72) m, so the base is at the same height as the table surface. That specific choice was made to allow the camera to sweep around the tabletop rather than only look across it.

Table 3: Robot-arm preset used at the checkpoint.

Parameter	Joint 1	Joint 2	Joint 3
Link lengths [m]	0.34	0.30	0.22
Initial servo angle [deg]	-36.0	60.0	-56.0
Min limit [deg]	-55.0	-12.0	-82.0
Max limit [deg]	55.0	86.0	76.0
Speed limit [deg/s]	66.0	88.0	108.0

The auto-demo is a smooth keyframed 20.000,0s sequence. It is not random motion: it was tuned to create repeated transitions between wall visibility, tabletop visibility, shallower views, and higher oblique views. That makes it useful for debugging both continuity-aware initialization and multi-tag world fitting.

Table 4: Auto-demo keyframes for the tabletop challenge.

Time [s]	Servo 1 [deg]	Servo 2 [deg]	Servo 3 [deg]
0.0	-36.0	60.0	-56.0
2.4	-20.0	76.0	-78.0
4.9	-4.0	54.0	-36.0
7.3	12.0	70.0	-72.0
9.8	6.0	34.0	-18.0
12.6	-16.0	80.0	-80.0
15.4	-34.0	58.0	-46.0
17.7	-10.0	66.0	-68.0
20.0	-36.0	60.0	-56.0

1.6 Camera Views and Data Products

The environment exposes one primary recorded phone view and three observer views. The primary view is the only one used for the checkpoint pose-estimation results; the observer views are for operator understanding and debugging.

Table 5: Camera views exposed by the browser app.

Camera	Resolution	Horizontal FOV	Position [m]	Purpose
Primary phone view	960 × 540	72.000,0°	robot-mounted	recorded and later re-analyzed
Room-wide observer	640 × 360	54.000,0°	(1.80, 2.90, 1.78)	whole-room debugging view
Side observer	640 × 360	50.000,0°	(−2.25, 1.55, 1.42)	lateral visibility of arm/table geometry
Top-oblique observer	640 × 360	46.000,0°	(0.20, 2.25, 2.18)	near top-down debugging view

Every recorded run stores both raw outputs and analysis products. That separation is important for calibration work: raw data remain the authoritative source, while analyses can be rerun as the estimator improves.

Table 6: Primary run artifacts stored for each recording.

Artifact	Meaning
phone_raw.mp4	recorded primary-camera video without analysis overlays
imu.csv	synthetic accel and gyro stream in the phone IMU frame
camera_gt.csv	camera ground-truth trajectory
samples.jsonl	per-tick state, detections, and exported measurement data
metadata.json	scene, camera, device, and run metadata
analysis/report.md	checkpoint-style markdown summary
analysis/pose_estimates.jsonl	per-observation single-tag solves
analysis/joint_world_estimates.jsonl	per-frame joint world solves

2 AprilTag Fiducial System

2.1 Why AprilTag is the Right Target Layer Here

The repository uses the AprilTag 36h11 family. This is a practical systems decision rather than a claim that no other fiducial family is viable. The main reasons are:

- AprilTag offers robust identification and low false-positive behavior for calibration-style scenes [7, 10],
- square planar fiducials are convenient for controlled pose-estimation studies and compare naturally with planar-pose references such as IPPE [4],
- OpenCV exposes a ready-to-run AprilTag detector path, which keeps the checkpoint easy to reproduce locally,
- newer work such as flexible tag layouts and low-frequency long-range tags still fits naturally into the same design space [6, 9].

Most importantly, the tags are not treated as abstract landmarks. They are rendered into the environment, then re-detected from the recorded video. That means the estimator sees image measurements produced by a detector, not perfect simulator correspondences.

2.2 How the Tags are Used in This Environment

The checkpoint environment uses two tag scales and two spatial roles:

- wall-mounted 0.100,0m tags define the room-scale frame and produce larger pose baselines,
- cube-top 0.060,0m tags make the camera solve meaningful in the near-field tabletop task where grasp-like viewpoints dominate.

This split is deliberate. If the scene only had far wall tags, the interesting tabletop views would not stress the estimator properly. If it only had close tabletop tags, the system would lose long-range global reference. Together, the tags produce a better calibration and trajectory-validation environment.

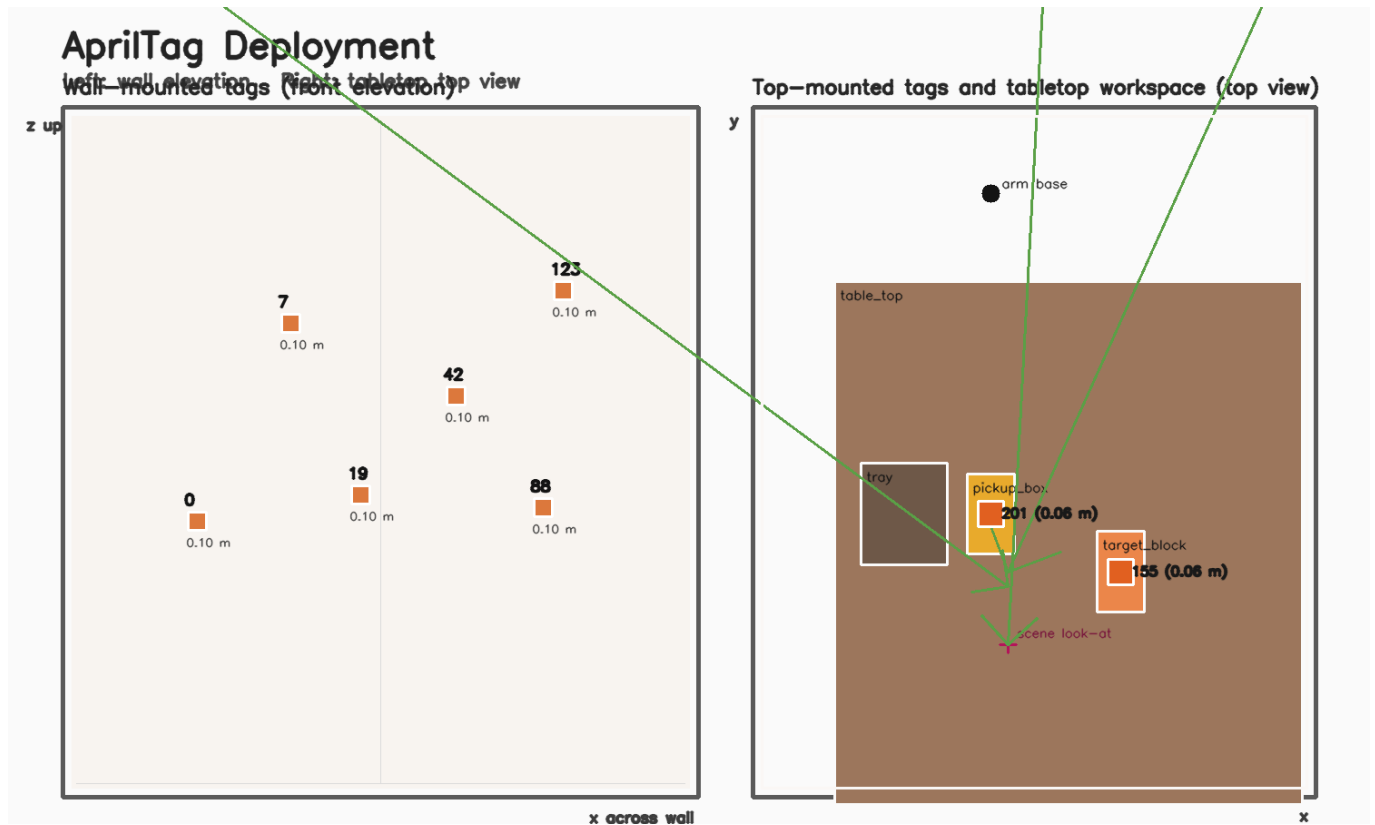


Figure 3: Configured AprilTag deployment for Checkpoint 01. Left: wall-mounted tags shown in wall elevation. Right: tabletop workspace and top-mounted tags in top view.

Figure 4 shows the same idea from the primary camera itself: large wall tags remain visible as global references while the smaller cube-top tags occupy the near field during the tabletop part of the motion.

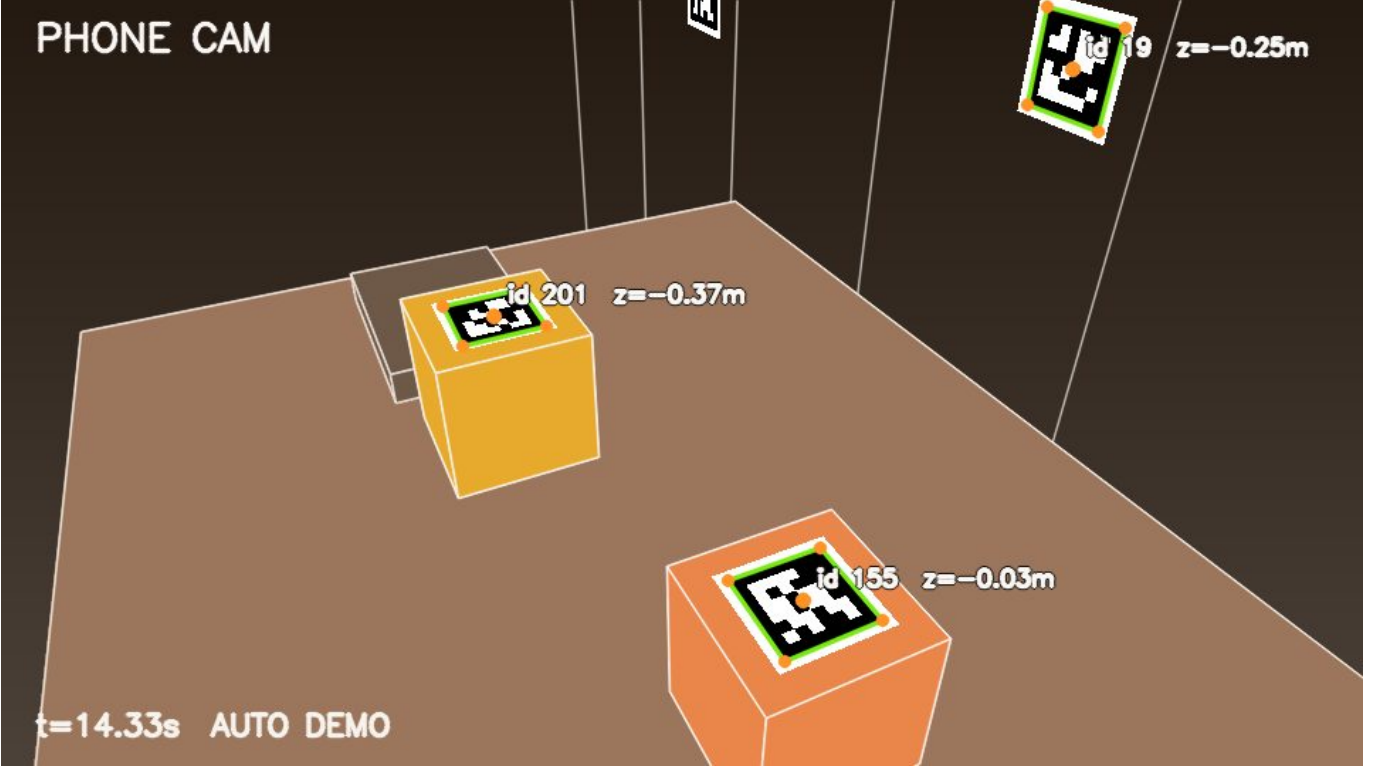


Figure 4: Example primary-camera frame showing the mixed-scale tag regime used in the checkpoint scene.

2.3 Configured Tag Inventory

Table 7 lists the full configured inventory. Only a subset is visible in every frame, which is intentional: intermittent visibility creates the track breaks needed to test continuity-aware initialization and reappearance handling.

Table 7: Configured fiducial tags in the checkpoint scene.

Tag ID	Family	Size [m]	Mount	World position [m]
0	36h11	0.10	wall	(−0.92, 0.00, 0.84)
7	36h11	0.10	wall	(−0.45, 0.00, 1.44)
19	36h11	0.10	wall	(−0.10, 0.00, 0.92)
42	36h11	0.10	wall	(0.38, 0.00, 1.22)
88	36h11	0.10	wall	(0.82, 0.00, 0.88)
123	36h11	0.10	wall	(0.92, 0.00, 1.54)
155	36h11	0.06	top	(0.30, 0.56, 0.806)
201	36h11	0.06	top	(0.00, 0.64, 0.846)

2.4 Detection Representation Used by the Solver

The OpenCV-based detector returns four corners in clockwise order. The repository then adds a fifth point: the projective center computed from the diagonal intersection. The optimizer always receives the same ordered five-point observation:

$$[\text{center}, \text{top_right}, \text{bottom_right}, \text{bottom_left}, \text{top_left}].$$

This explicit ordering matters because it keeps the measurement convention stable across detector, exporter, and optimizer. It also mirrors the local pose-estimation tooling used elsewhere in the repository.

For a tag half-extent h , the corresponding object points are

$$\mathbf{P}_0 = (0, 0, 0), \mathbf{P}_1 = (h, -h, 0), \mathbf{P}_2 = (h, h, 0), \mathbf{P}_3 = (-h, h, 0), \mathbf{P}_4 = (-h, -h, 0).$$

2.5 Why the Tag Section Comes Before the Camera Model

This report now introduces the tags before the camera details on purpose. The estimator does not operate on pixels in the abstract; it operates on image measurements of known planar objects. So the tag geometry, size conventions, and ordering are logically prior to the camera model and the optimization method.

3 Checkpoint Scope and Repository State

Checkpoint 01 corresponds to commit `final IMU logging fix checkpoint` and to the preserved analysis report derived from the recorded run `run_20260406_083611`. At this checkpoint, the repository has a clear primary path:

- a browser-based interactive simulation and recording application is fully runnable,
- a Pixel 9a-inspired camera model is configured and verified against the recorded-phone assumptions used in this repository,
- raw video, IMU-like telemetry, camera ground truth, and per-frame metadata can be recorded,
- AprilTag-based re-detection and pose estimation can be replayed offline,
- continuity-aware single-tag fitting and joint all-points world-trajectory fitting are both working at checkpoint quality,
- the corrected recording path now preserves exact IMU/time values and exact logged start velocity well enough for effectively exact inertial dead reckoning on the preserved run.

This is intentionally not yet a full Isaac Sim production stack. The richer Isaac/ROS runtime remains scaffold-level in the repository, while the browser simulator is the current executable reference implementation.

3.1 Checkpoint Results at a Glance

Table 8 summarizes the preserved metrics for the checkpoint run. The first block is the aggregate over all single-tag solves. The second block is the joint world-frame solve per frame, where all visible tag points are optimized together. The third block is the corrected IMU-only dead-reckoning result after exact IMU/time persistence and exact logged initial velocity were added to the recording path.

Table 8: Checkpoint 01 headline metrics from run `run_20260406_083611`.

Metric	Single-tag solve	Joint all-points solve	IMU-only trajectory
Frames analyzed	240	240	240
Solved observations	712	240 frame-level solves	239 propagated steps
Mean position error [m]	0.0042	0.0006	2.16×10^{-7}
Median position error [m]	0.0023	0.0003	1.43×10^{-7}
Max position error [m]	0.1299	0.0063	7.15×10^{-7}
Mean rotation error [deg]	0.2004	0.0806	0.0013
Mean reprojection RMSE [px]	0.0457	0.0843	n/a

The important interpretation is that the single-tag pipeline is already accurate enough for detailed per-tag analysis, the joint all-points trajectory is strong enough to serve as a trusted visual reference inside the simulator, and the corrected IMU path is now accurate enough to serve as a synthetic inertial baseline for future visual-inertial work.

4 Camera and IMU Model

4.1 Source of the Phone Parameters

Checkpoint 01 uses the preset `config/camera/pixel_9a_main.toml` together with the device profile `config/device/pixel_9a_phone.yaml`. These values were tuned from a real Pixel 9a metadata dump collected by the companion recording workflow, then adjusted so the simulator better matches the processed MP4 output of that phone rather than an uncorrected raw sensor path.

The underlying Camera2 concepts follow the Android camera metadata definitions for lens intrinsics and distortion [1, 2].

4.2 Active-Array and Output Geometry

The stored phone metadata describe a 4000×3000 active array. The checkpoint video output is 1920×1080 . The repository therefore models the phone video as a center crop of the active array into a 16:9 aspect ratio, followed by a scale to the output resolution.

For the active-array intrinsics (f_x, f_y, c_x, c_y) and crop rectangle (x_c, y_c, w_c, h_c) , the effective video intrinsics are

$$f'_x = f_x \frac{W_o}{w_c}, \quad f'_y = f_y \frac{H_o}{h_c}, \quad c'_x = (c_x - x_c) \frac{W_o}{w_c}, \quad c'_y = (c_y - y_c) \frac{H_o}{h_c}.$$

For the current preset, the active aspect ratio is narrower than 16:9, so the crop keeps the full width and trims the height. Numerically:

- active array: 4000×3000 ,
- crop rectangle: $(x_c, y_c, w_c, h_c) = (0, 375, 4000, 2250)$,
- output scale: 0.48 in both axes,
- effective output intrinsics: $f_x = f_y = 1293.17136$, $c_x = 960.292272$, $c_y = 543.63216$.

Table 9: Primary phone camera parameters used for the checkpoint run.

Parameter group	Value
Device	Google Pixel 9a, logical camera 0
Sensor active array	4000×3000 pixels
Output video mode	1920×1080 pixels, center crop
Sensor orientation	90°
Focal length	4.530,0 mm
Focus distance	0.2650956 diopters
Target frame rate	30.000,0 Hz
Timestamp source	REALTIME
Active-array intrinsics	$f_x = f_y = 2694.107$, $c_x = 2000.6089$, $c_y = 1507.567$
Effective output intrinsics	$f_x = f_y = 1293.17136$, $c_x = 960.292272$, $c_y = 543.63216$
Projection preset	<code>pixel_processed_video_default</code>
Render-time lens distortion	disabled for checkpoint 01

4.3 Distortion Model

The preset keeps OpenCV Brown–Conrady coefficients

$$[k_1, k_2, k_3, p_1, p_2] = [0.15084998, -0.44805366, 0.38709834, 0.0, 0.0].$$

For normalized ideal coordinates (x, y) and radius $r^2 = x^2 + y^2$, the standard polynomial form is

$$x_d = x \left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 \right) + 2p_1 xy + p_2 (r^2 + 2x^2), \quad (1)$$

$$y_d = y \left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 \right) + p_1 (r^2 + 2y^2) + 2p_2 xy. \quad (2)$$

At Checkpoint 01 these coefficients are preserved in the configuration and available to the offline analysis, but the rendered video path does *not* apply additional lens warping. This is intentional: the repository previously tested a more strongly distorted render mode and found it mismatched the actual processed-video appearance of the phone. The checkpoint therefore models the processed MP4 path rather than a raw pre-correction sensor image.

4.4 Image-Plane Measurement Convention

After detection, pixel points are converted into normalized camera coordinates and then into image-plane metric coordinates using the focal length:

$$x_m = f \frac{u - c_x}{f_x}, \quad y_m = -f \frac{v - c_y}{f_y},$$

where f is the physical focal length in meters and (u, v) are pixel coordinates.

The sign convention on y makes the exported image-plane coordinates use an upward-positive axis, which matches the estimator implemented in this repository.

4.5 IMU Model

The checkpoint keeps a lightweight but explicit IMU profile attached to the phone. It is not yet a full bias-random-walk inertial simulator, but after the final logging fix it is sufficient to preserve the simulator accel/gyro/time stream exactly enough for essentially exact inertial dead reckoning on the preserved checkpoint run.

Table 10: Phone IMU parameters used by the checkpoint device preset.

Parameter	Value
Logged checkpoint rate	12.000,0 Hz (one sample per recorded simulation frame)
Gravity included	yes
Accel white-noise std	zero in the preserved synthetic run
Gyro white-noise std	zero in the preserved synthetic run
Accel bias	zero at checkpoint
Gyro bias	zero at checkpoint
IMU translation from camera	(0, 0, 0.01) m
IMU orientation relative to camera	(0, 0, 0)°

5 Offline Analysis and Pose Fitting

5.1 Pipeline Overview

The checkpoint analysis replays the recorded run offline and performs the following steps:

1. load the run metadata, samples, and raw phone video,
2. choose whether the recorded image needs undistortion,
3. run AprilTag detection on the recorded frames,
4. export each detection into the five-point image-plane measurement vector,

5. estimate per-tag camera pose from image measurements alone,
6. estimate a joint world-frame camera pose from all visible tag points together,
7. compare both families of estimates against the recorded simulator ground truth,
8. write visual and numerical artifacts under the run’s `analysis/` directory.

5.2 Single-Tag State and Objective

For a single tag, the optimized state is

$$\mathbf{x} = \begin{bmatrix} t_x & t_y & t_z & r_x & r_y & r_z \end{bmatrix}^\top,$$

where (t_x, t_y, t_z) is the tag-to-camera translation in meters and (r_x, r_y, r_z) is a Rodrigues rotation vector.

Let $\mathbf{P}_i$ be the five object points on the tag plane and let $\hat{\mathbf{m}}_i$ be the observed image-plane points. The prediction function is

$$\mathbf{m}_i(\mathbf{x}) = f \frac{(\mathbf{R}(\mathbf{r})\mathbf{P}_i + \mathbf{t})_{xy}}{(\mathbf{R}(\mathbf{r})\mathbf{P}_i + \mathbf{t})_z}.$$

The loss minimized by the solver is

$$\mathcal{L}(\mathbf{x}) = \frac{1}{2} \sum_i \|\mathbf{m}_i(\mathbf{x}) - \hat{\mathbf{m}}_i\|_2^2 + \lambda \sum_i \min(z_i - \epsilon, 0)^2,$$

where the second term penalizes non-positive predicted depth.

5.3 Initialization Policy

The checkpoint no longer uses a single global seeding rule. Instead it uses a continuity-aware initialization policy:

- the first frame of a visible tag streak uses a camera-obscure seed derived from apparent tag scale and center offset,
- subsequent frames in the same streak warm-start from the previous frame’s solution for that same tag,
- if a tag reappears after disappearing, the solver can also fall back to a previous world-frame estimate from the frame-level joint fit.

This was an important checkpoint improvement: earlier versions seeded every frame from a fresh pinhole guess, which made the planar single-tag problem unnecessarily brittle.

5.4 Modified Newton Solver

Gradients and Hessians are computed with JAX [3]. The checkpoint solver uses a modified Newton step rather than plain undamped Newton:

$$(\mathbf{H} + \mu \mathbf{I})\mathbf{p} = -\mathbf{g}.$$

Here $\mathbf{g}$ is the gradient, $\mathbf{H}$ the Hessian, and μ a diagonal shift. The solver starts from the raw Hessian step and, if the step is singular or not a descent direction, increases μ until a valid descent step is recovered. An Armijo backtracking line search then chooses a step length α and updates

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha \mathbf{p}_k.$$

This specific solver choice matters because one of the main pre-checkpoint bugs was not in the camera model but in the optimization step selection: some Newton steps were being silently replaced by an almost-zero gradient fallback, which prevented motion even when the optimizer was not in a meaningful minimum.

5.5 Joint World Solve

In addition to per-tag solves, the pipeline performs a per-frame joint optimization over all visible points transformed into a shared world frame. This produces a much stronger trajectory estimate and serves as the most accurate end-of-run reference in the checkpoint artifacts.

Conceptually, this is closer to a multi-target calibration observation than to a pure one-tag pose problem. It reduces the planar ambiguity present in single-tag solves and is the main reason the joint trajectory errors are much smaller than the aggregated single-tag errors.

5.6 Relation to Common References

The repository’s estimator is not a direct implementation of IPPE, Kalibr, or any single canonical package. Instead it is a project-specific optimizer informed by those lines of work:

- Zhang calibration gives the standard planar-camera-calibration background [11],
- IPPE is a strong reference point for planar-pose ambiguity and efficiency [4],
- camera–IMU calibration pipelines such as Kalibr motivate the broader structure of synchronized camera and inertial metadata in a calibration system [5, 8],
- JAX supplies the practical automatic-differentiation machinery for gradients and Hessians [3].

6 Checkpoint 01 Results

6.1 Per-Tag Summary

Only four configured tags contribute heavily to the checkpoint metrics because they dominate the visible portions of the motion. Table 11 lists the verified per-tag statistics from the preserved run.

Table 11: Per-tag pose-estimation results at Checkpoint 01.

Tag	Obs.	Mean pos. [m]	Median pos. [m]	Max pos. [m]	Mean rot. [deg]	Mean reproj. [px]
0	95	0.0095	0.0090	0.0211	0.3069	0.0628
19	180	0.0066	0.0033	0.1299	0.3044	0.0594
155	240	0.0016	0.0013	0.0102	0.1272	0.0333
201	197	0.0025	0.0021	0.0090	0.1434	0.0401

The best overall per-tag behavior comes from the tabletop cube-top tags (155 and 201), which are closer to the camera and usually occupy a larger portion of the image. Tag 19 is the only tag with a clearly visible tail in the maximum position error, reaching 0.129,9 m on its worst observation; this is still below the earlier meter-scale failures seen before the solver corrections.

6.2 Representative Diagnostic Views

Figure 5 and Figure 6 are the core checkpoint diagnostics copied from the preserved run. They show that the checkpoint is not only numerically good on average but also visually self-consistent in a representative solved frame.

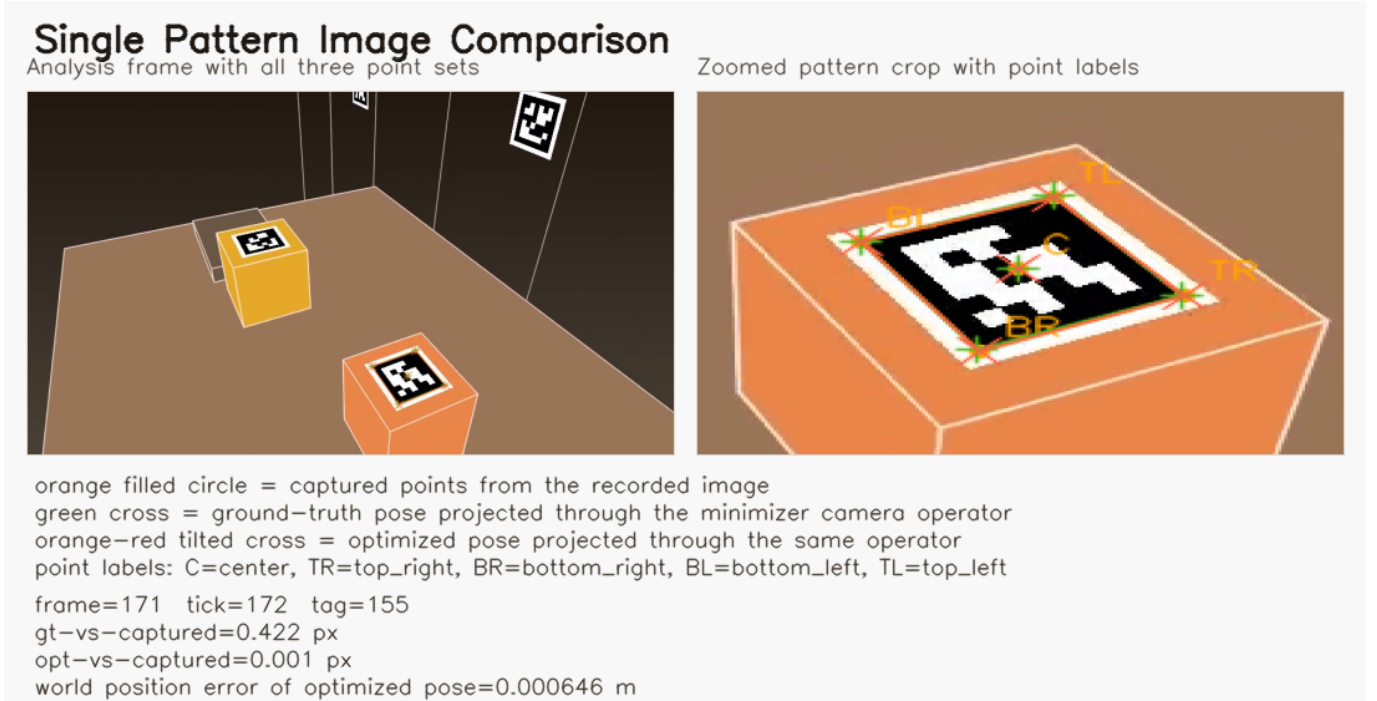


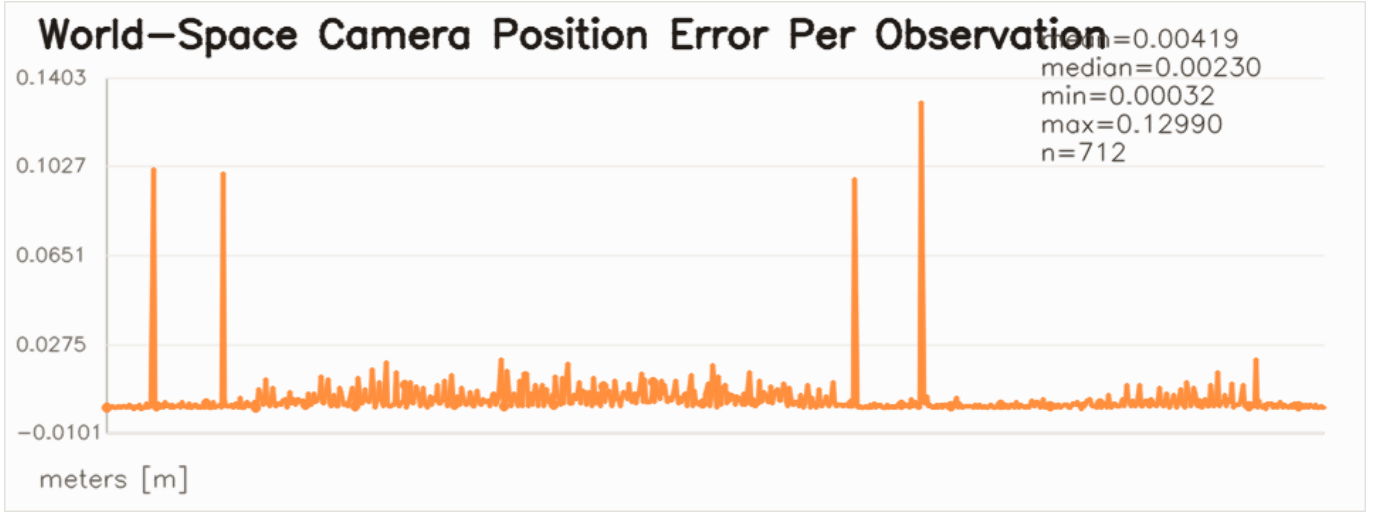
Figure 6: Representative single-pattern overlay. Orange circles are detected image points, green marks show the camera operator at ground truth, and orange-red tilted crosses show the final fitted result.

6.3 Global Error Timelines

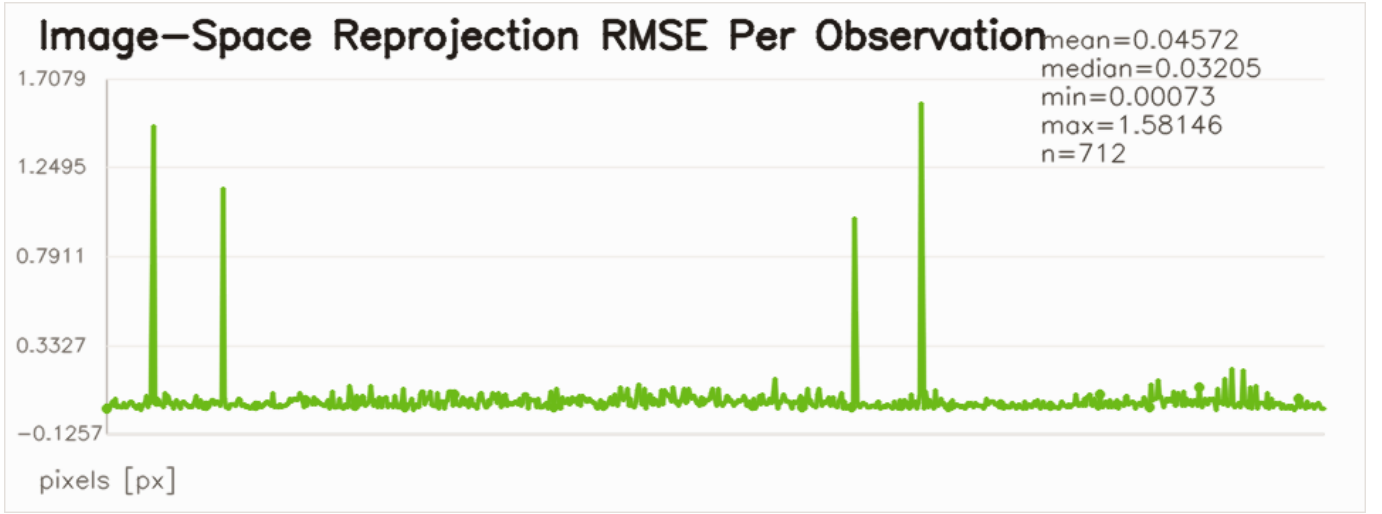
Figure 7 shows the two key scalar diagnostics:

- world-space camera-position error in meters,
- image-space reprojection RMSE in pixels.

The difference between these units matters. Small reprojection error does not automatically imply small world-space error in a planar single-tag problem, but by Checkpoint 01 both have been brought into a plausible regime.



(a) World-space camera-position error [m].



(b) Image-space reprojection RMSE [px].

Figure 7: Checkpoint-wide scalar error diagnostics.

6.4 Optimizer Trace

Figure 8 is the preserved representative optimizer loss trace. It exists mainly as evidence that the corrected solver is descending as intended and no longer exhibits the broken “no-motion” behavior that triggered the earlier investigation cycle.

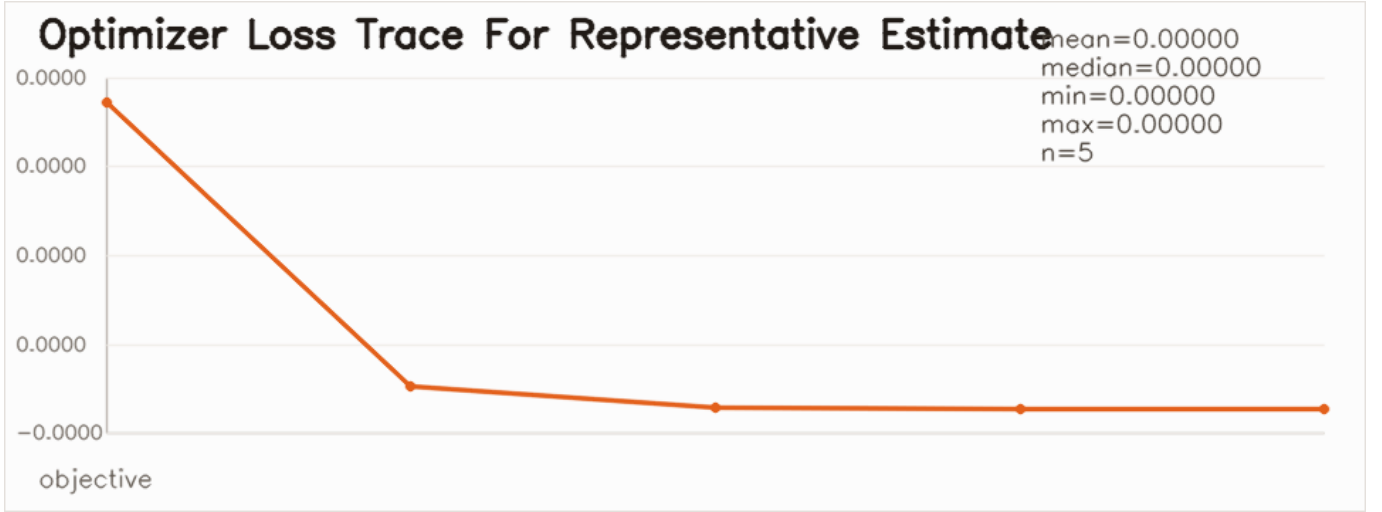


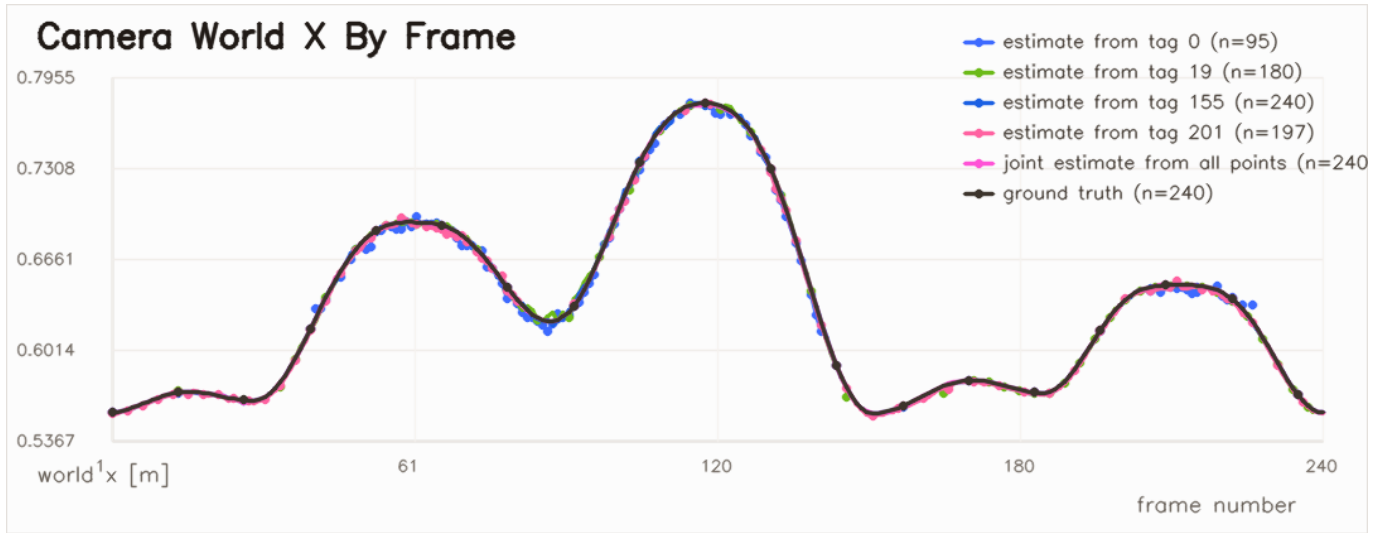
Figure 8: Representative optimizer loss trace from the checkpoint run.

6.5 World-Trajectory Comparison

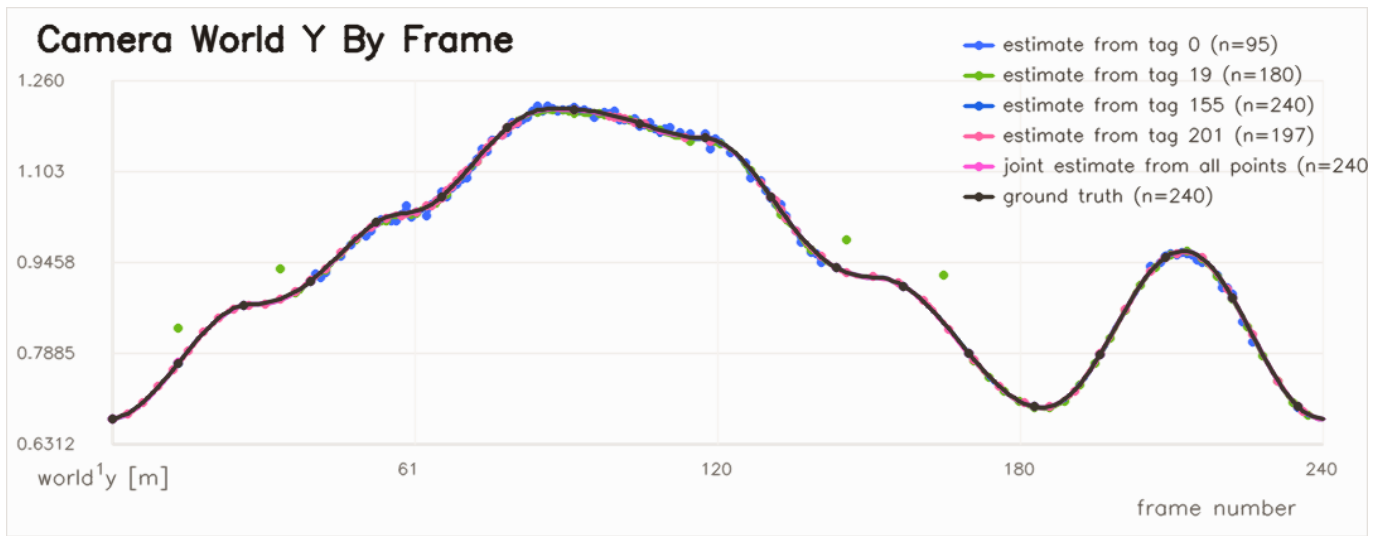
The six component plots in Figure 9 and Figure 10 compare, over frame index:

- the independently reconstructed per-tag world-frame trajectories,
- the joint all-points world solution,
- the recorded simulator ground truth.

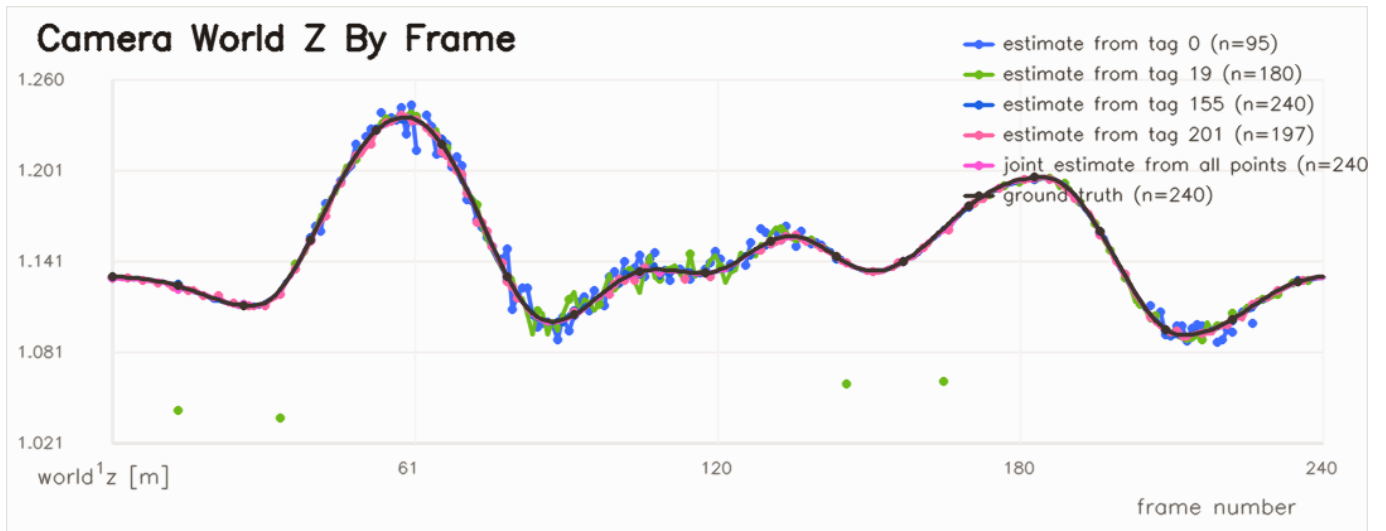
They are useful because they reveal consistency of the recovered trajectory, not only endpoint error statistics.



(a) x [m]

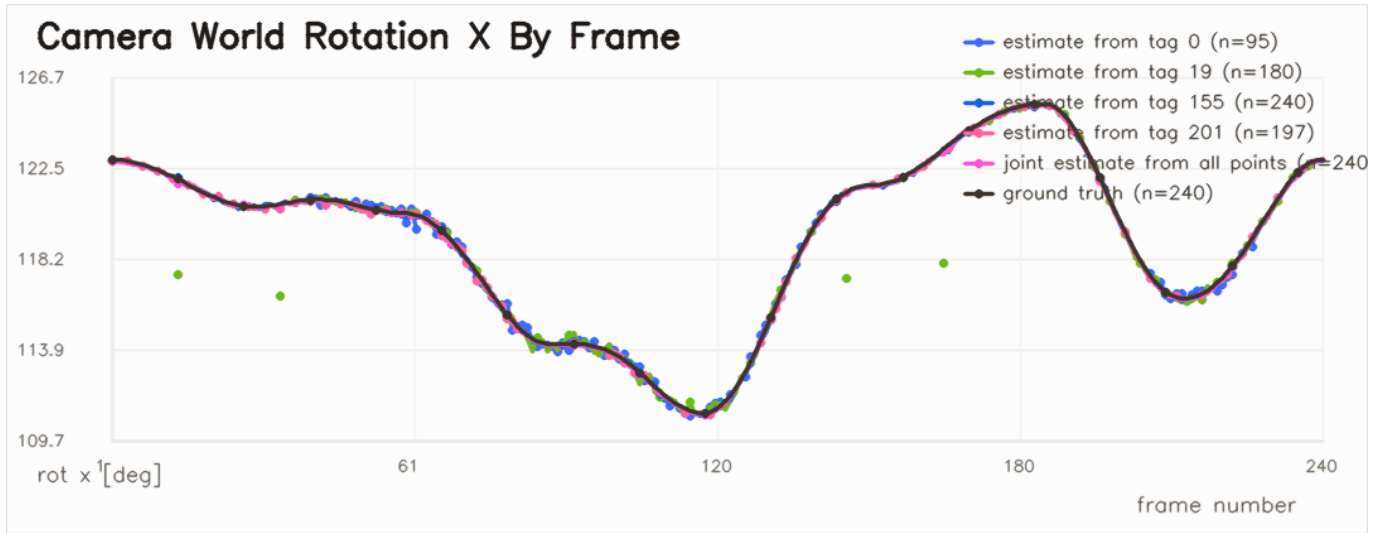


(b) y [m]

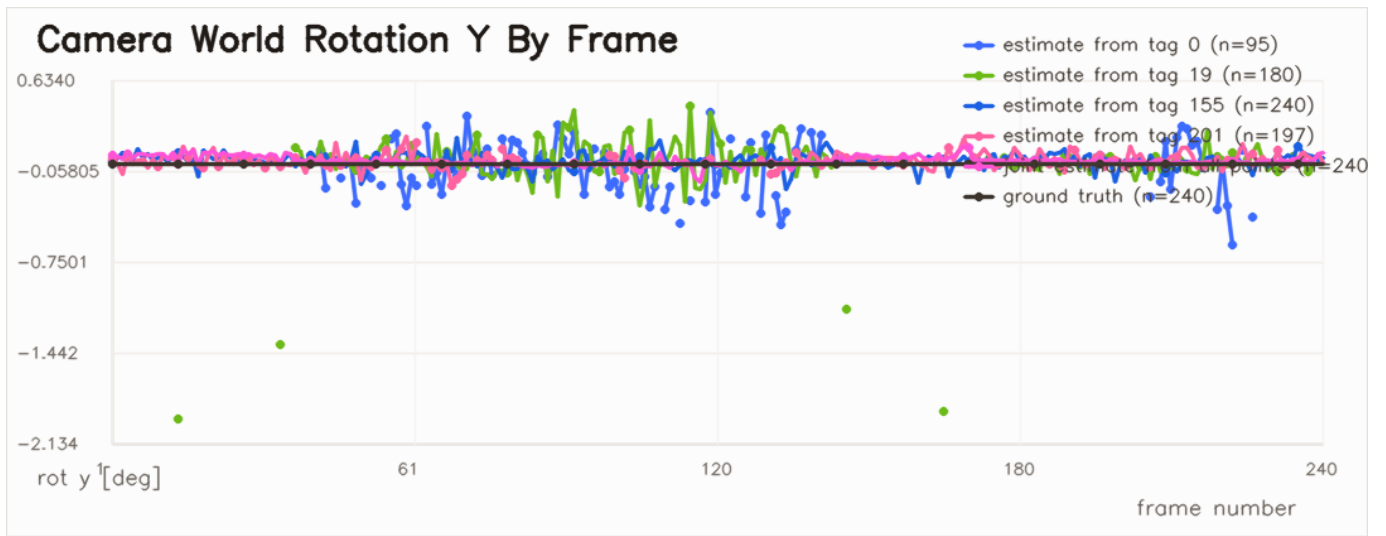


(c) z [m]

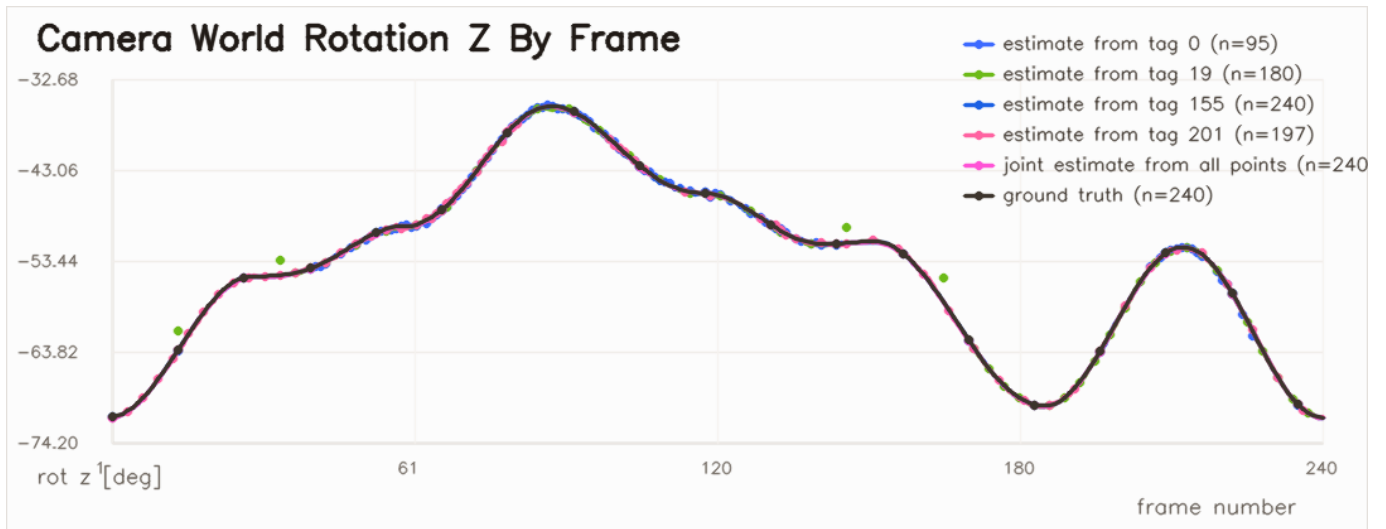
Figure 9: World-frame position components over frame number.



(a) r_x [deg]



(b) r_y [deg]



(c) r_z [deg]

Figure 10: World-frame rotation components over frame number.

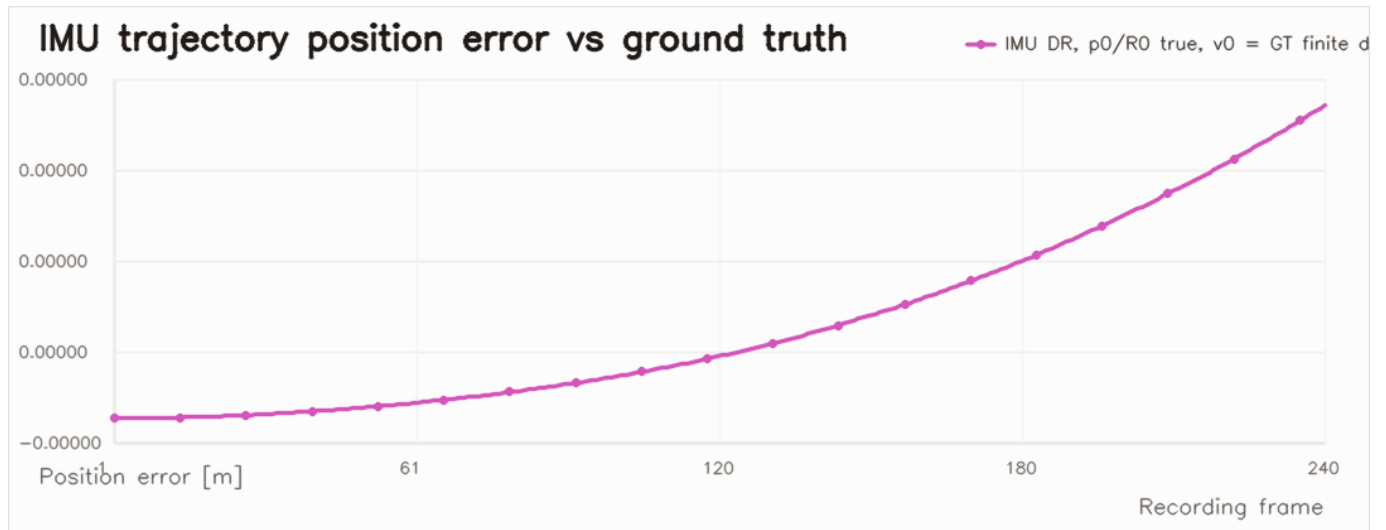
6.6 IMU-Only Trajectory Reconstruction

The checkpoint now also includes a corrected inertial-only reconstruction driven entirely by the recorded accelerometer and gyroscope stream. The first camera world pose is fixed to the true value, and the initial world velocity is read directly from the logged `camera_gt.csv` trajectory.

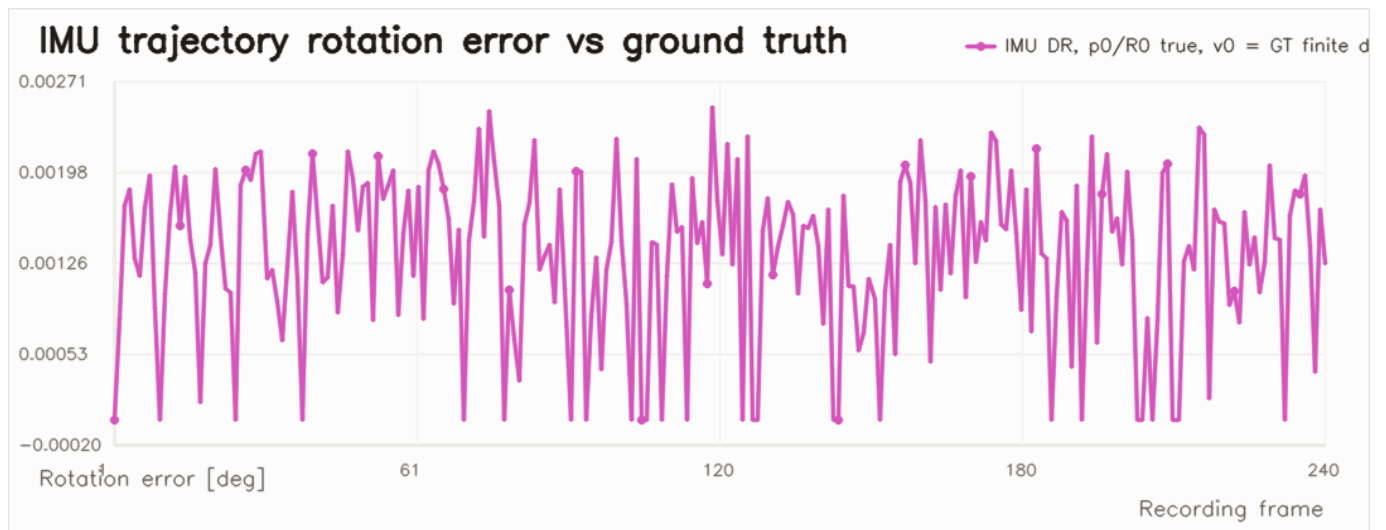
This detail matters because the immediately preceding legacy run stored rounded IMU values and did not log the exact initial world velocity. That older path produced visible translation drift despite the simulator being deterministic. After fixing the recorder and matching the simulator’s discrete inertial update convention in the analysis, the IMU-only trajectory became essentially exact on the preserved run.

Table 12: IMU-only trajectory metrics from run `run_20260406_083611`.

Metric	Value
Initial velocity source	<code>logged_camera_gt_velocity</code>
Mean position error [m]	2.1622×10^{-7}
Median position error [m]	1.4336×10^{-7}
Max position error [m]	7.1542×10^{-7}
Mean rotation error [deg]	0.0013
Max rotation error [deg]	0.0025

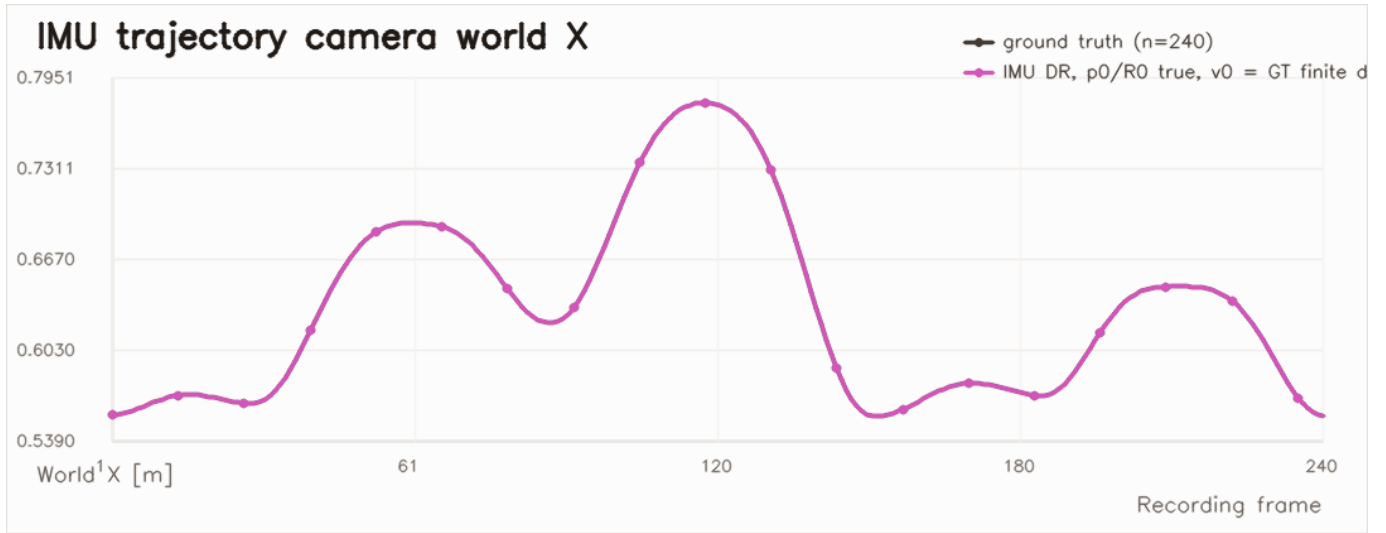


(a) IMU-only world-position error [m].

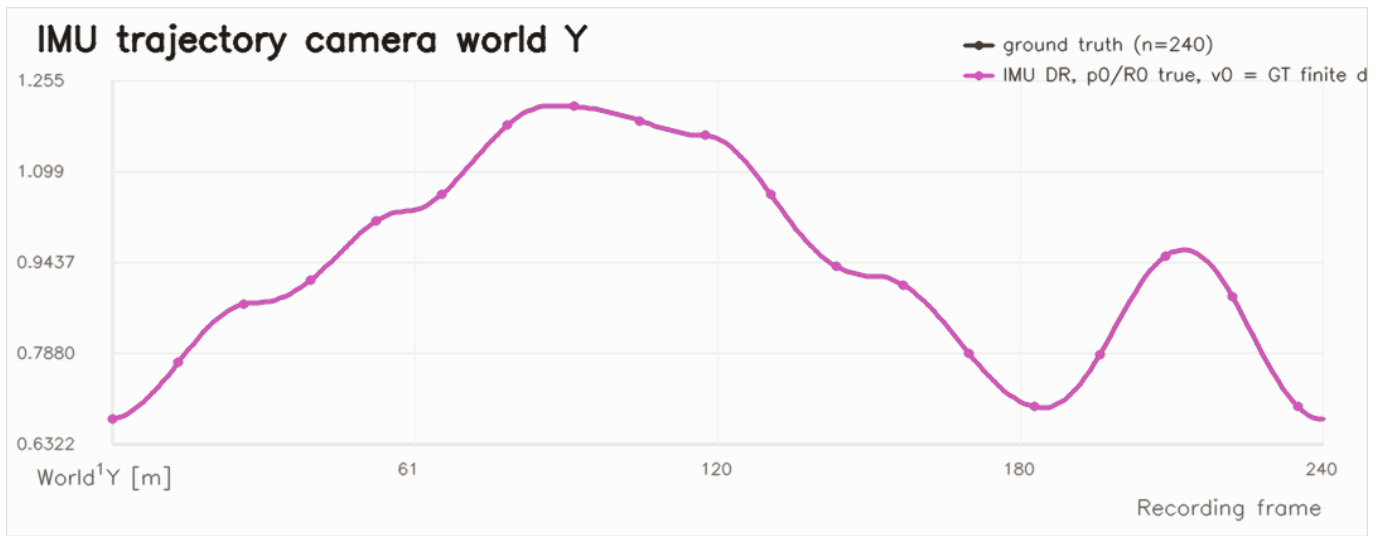


(b) IMU-only rotation error [deg].

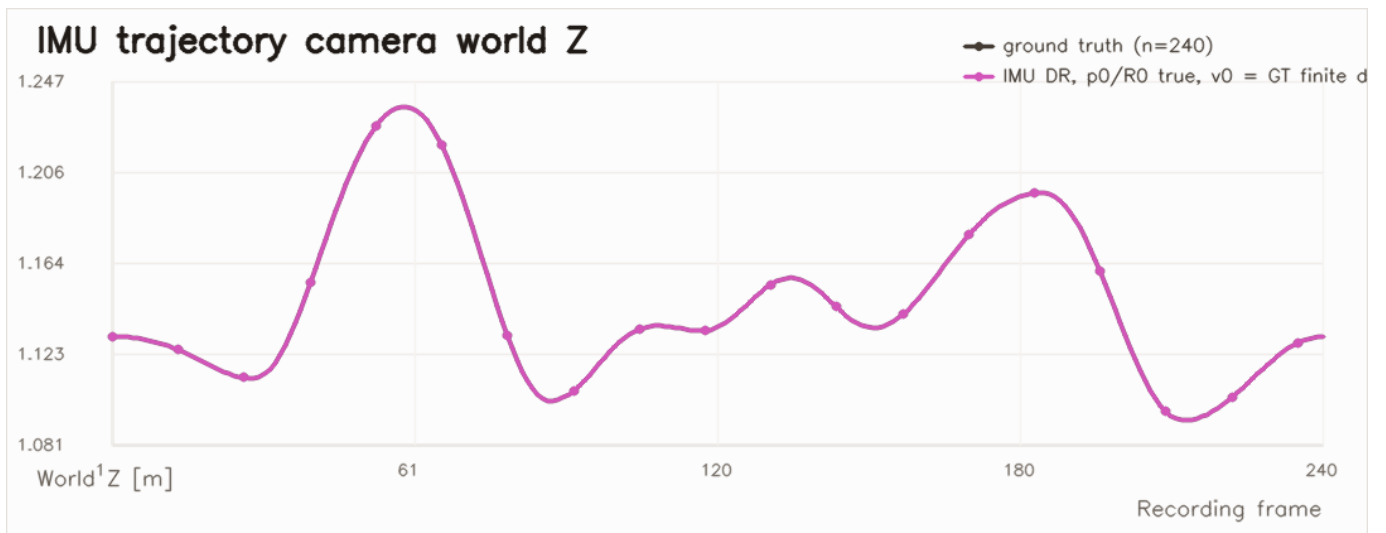
Figure 11: Scalar IMU-only error diagnostics after the exact logging fix.



(a) x [m]

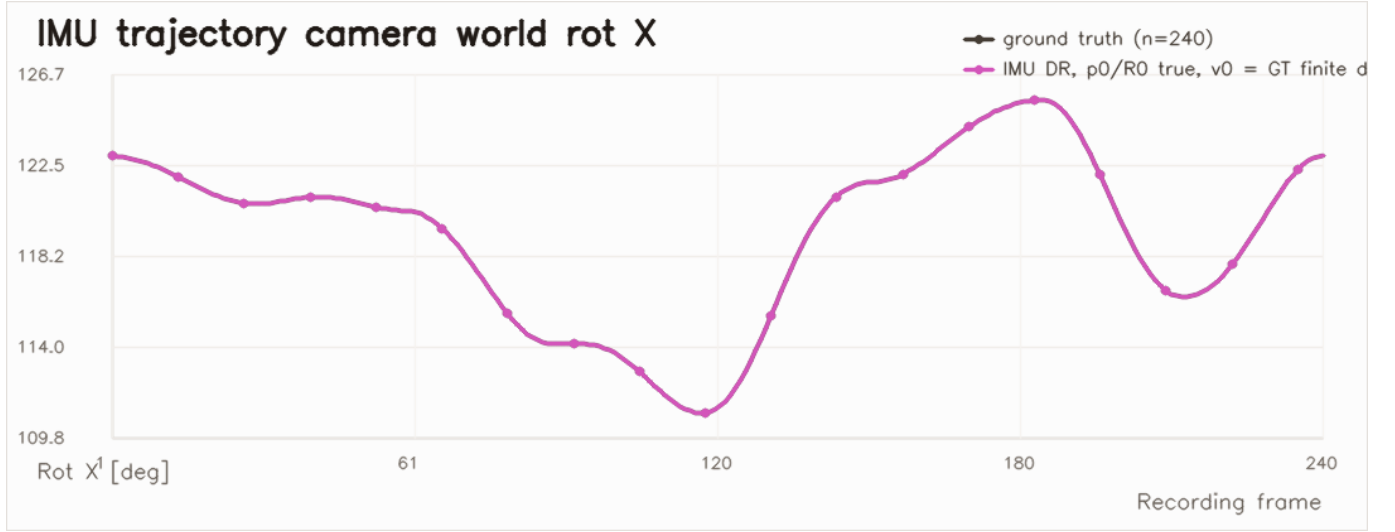


(b) y [m]

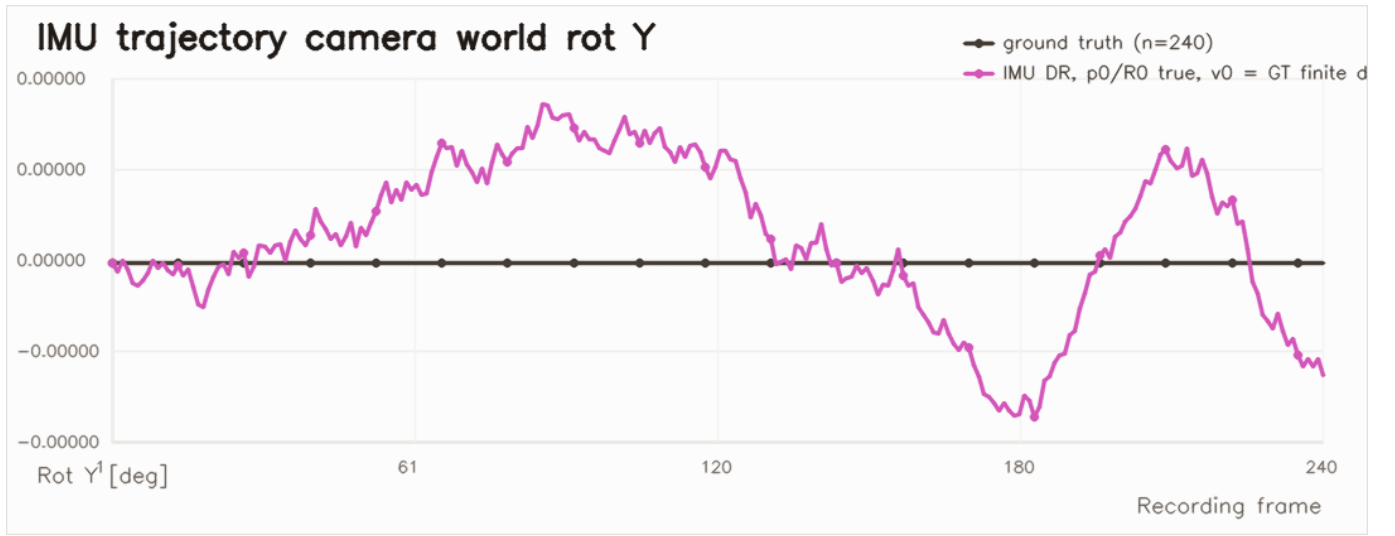


(c) z [m]

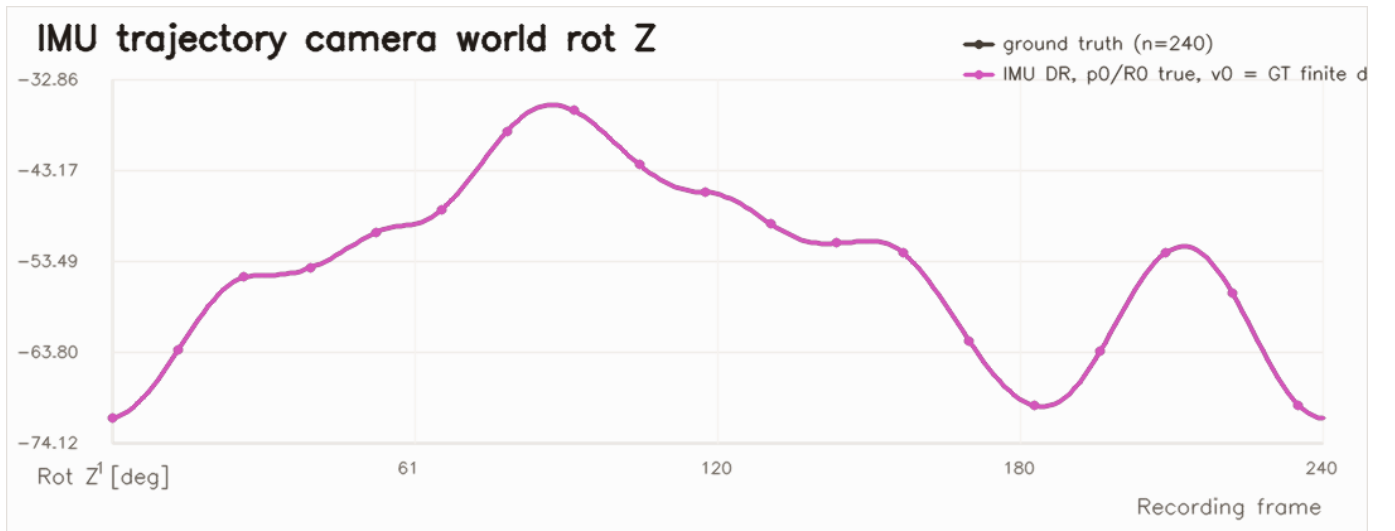
Figure 12: IMU-only world-frame position components over frame number.



(a) r_x [deg]



(b) r_y [deg]



(c) r_z [deg]

Figure 13: IMU-only world-frame rotation components over frame number.

6.7 Interpretation of the Checkpoint

The current checkpoint can be summarized in three practical statements:

1. The browser simulator is no longer only a qualitative demo; it now produces quantitatively useful camera-pose estimation experiments.
2. The main failure mode discovered during development was an optimizer-step-selection bug, not a broken forward camera model.
3. The corrected IMU recording path is now accurate enough that inertial-only trajectory reconstruction can also be used as a trustworthy synthetic reference.

This is why Checkpoint 01 was frozen and copied into the repository documentation. It is the first state where the app, the recorder, the camera model, the tag detector, and the offline estimator all agree well enough to preserve as a stable baseline.

7 Limitations and Next Steps

Even though Checkpoint 01 is strong enough to preserve, it remains a checkpoint rather than an endpoint. The main remaining limitations are:

- the simulator is still geometry-driven and browser-oriented, not a full photorealistic physics environment,
- the IMU model is now logged exactly enough for the preserved run, but it is still simple compared with full stochastic inertial simulation,
- the processed-video camera path is modeled directly, while a raw-sensor path exists only as an alternate preset,
- the Isaac Sim integration in the repository is still scaffold-level and not part of the validated checkpoint path.

The immediate forward path after this checkpoint is therefore not “make the current pipeline basically work”—that part is achieved—but rather:

- extend scene richness and dynamic complexity,
- strengthen the sensor models,
- connect the validated browser-side workflow to the heavier Isaac/ROS stack when that stack is ready to be real rather than aspirational.

8 Conclusion

Checkpoint 01 is a self-consistent, reproducible state of the repository. It provides:

- a documented synthetic scene and motion program,
- a concrete fiducial inventory,
- a Pixel 9a-derived camera and IMU model,
- a verified image-only pose-estimation pipeline,
- a verified IMU-only trajectory reconstruction path on exact persisted simulator telemetry,
- preserved figures and metrics that can be sent to collaborators as a stable reference.

For a project that began as a scaffold around a future digital twin, this checkpoint matters because it turns the repository into something concretely inspectable and quantitatively testable today.

References

- [1] Android Developers. Cameracharacteristics.lens_distortion. Android Camera2 API reference, 2026. Accessed 2026-04-05. URL: https://developer.android.com/reference/android/hardware/camera2/CameraCharacteristics#LENS_DISTORTION.
- [2] Android Developers. Cameracharacteristics.lens_intrinsic_calibration. Android Camera2 API reference, 2026. Accessed 2026-04-05. URL: https://developer.android.com/reference/android/hardware/camera2/CameraCharacteristics#LENS_INTRINSIC_CALIBRATION.
- [3] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of python+numpy programs. Software repository, 2018. Version 0.3.13 citation entry from the official repository. URL: <https://github.com/google/jax>.
- [4] Toby Collins and Adrien Bartoli. Infinitesimal plane-based pose estimation. *International Journal of Computer Vision*, 109(3):252–286, 2014. doi: 10.1007/s11263-014-0725-5.
- [5] Paul Furgale, Joern Rehder, and Roland Siegwart. Unified temporal and spatial calibration for multi-sensor systems. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1280–1286, 2013. doi: 10.1109/IROS.2013.6696523.
- [6] Maximilian Krogus, Acshi Haggenmiller, and Edwin Olson. Flexible layouts for fiducial tags. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, October 2019. APRIL Lab paper page: <https://april.eecs.umich.edu/papers/details.php?name=krogus2019iros>.
- [7] Edwin Olson. AprilTag: A robust and flexible visual fiducial system. In *2011 IEEE International Conference on Robotics and Automation*, pages 3400–3407, May 2011. APRIL Lab paper page: <https://april.eecs.umich.edu/papers/details.php?name=olson2011tags>.
- [8] Joern Rehder and Roland Siegwart. Camera/IMU calibration revisited. *IEEE Sensors Journal*, 17(11): 3257–3268, 2017. doi: 10.1109/JSEN.2017.2674307.
- [9] Ben Wang. LFTag: A scalable visual fiducial system with low spatial frequency. arXiv:2006.00842 [cs.CV], 2020. URL: <https://arxiv.org/abs/2006.00842>.
- [10] John Wang and Edwin Olson. AprilTag 2: Efficient and robust fiducial detection. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, October 2016. APRIL Lab paper page: <https://april.eecs.umich.edu/papers/details.php?name=wang2016iros>.
- [11] Zhengyou Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330–1334, 2000. doi: 10.1109/34.888718.